4(1).Classification Model Using KNN and Evaluation Metrics

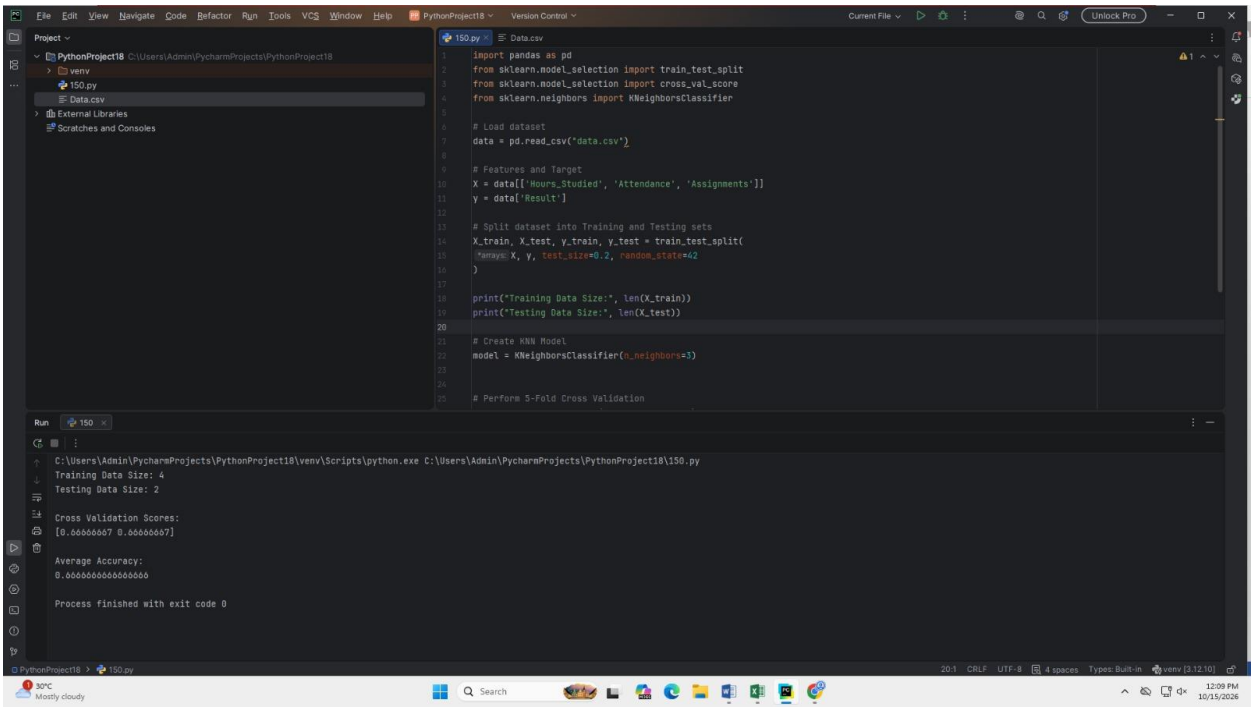
Aim :

To write a program that implements a classification model using the K-Nearest Neighbors (KNN) algorithm, trains it on a training set, predicts the class labels for the test set, and evaluates its performance using the confusion matrix, accuracy, precision, recall and F1-score.

Algorithm :

1. Start the program.
2. Import the required libraries (pandas, train_test_split from sklearn.model_selection, KNeighborsClassifier from sklearn.neighbors, and accuracy_score, precision_score, recall_score, f1_score, confusion_matrix and classification_report from sklearn.metrics).
3. Load the dataset from the input file.
4. Separate the data into input features (X) and the target variable (y).
5. Split the data into training and testing sets using train_test_split (for example 80% training and
20% testing), with a fixed random_state.
6. Create the KNeighborsClassifier model with a chosen number of neighbors k (for example k =
3).
7. Train the model by fitting it on the training data (X_train, y_train).
8. Predict the class labels for the test set (X_test).
9. For each test record, the model finds its k nearest training records (using Euclidean distance)
and assigns the majority class among them.
10. Compare the predicted labels with the actual labels (y_test) and build the confusion matrix
(TP, TN, FP, FN).
11. Calculate the evaluation metrics: accuracy, precision, recall and F1-score.
12. Display the confusion matrix and the evaluation metrics.
13. Stop the program.

Program&Result:



4(2).Classification Model Using SVM and Evaluation Metrics

Aim :

To write a program that implements a classification model using the Support Vector Machine (SVM) algorithm, trains it on a training set, predicts the class labels for the test set, and evaluates its performance using the confusion matrix, accuracy, precision, recall and F1-score.

Algorithm :

1. Start the program.
2. Import the required libraries (pandas, train_test_split from sklearn.model_selection, SVC from
sklearn.svm, and accuracy_score, precision_score, recall_score, f1_score, confusion_matrix and
classification_report from sklearn.metrics).
3. Load the dataset from the input file.
4. Separate the data into input features (X) and the target variable (y).
5. Split the data into training and testing sets using train_test_split (for example 80% training and

20% testing), with a fixed random_state.
6. Create the SVC model with a chosen kernel (for example linear or rbf) and regularization
parameter C.
7. Train the model by fitting it on the training data (X_train, y_train).
8. The model finds the optimal hyperplane that separates the classes with the maximum margin,
using the support vectors.
9. Predict the class labels for the test set (X_test).
10. Compare the predicted labels with the actual labels (y_test) and build the confusion matrix
(TP, TN, FP, FN).
11. Calculate the evaluation metrics: accuracy, precision, recall and F1-score.
12. Display the confusion matrix and the evaluation metrics.
13. Stop the program.

Program&Result:

